Desafio 16: ESP32 no Wokwi, do primeiro acesso à entrega

Este guia acompanha o projeto de monitoramento de temperatura de uma sala de servidores. A implementação foi criada a partir dos requisitos do desafio fornecido. Leia as explicações, execute a simulação e pratique as alterações de temperatura: o professor poderá pedir uma demonstração ao vivo. A orientação do enunciado é desenvolver a solução a partir dos requisitos; use o material como apoio de estudo e adapte conforme as regras da disciplina.

1. O que você vai construir

Um sensor DHT22 informa a temperatura ao ESP32. O programa decide a faixa de temperatura, acende um único LED e mostra os dados no monitor serial a cada 2 segundos. O ESP32 também se conecta ao Wi-Fi simulado para cumprir o desafio extra.

Temperatura	Estado	LED aceso
Menor que 25 °C	NORMAL	Verde
De 25 °C até 30 °C, inclusive	ATENÇÃO	Amarelo
Maior que 30 °C	CRÍTICO	Vermelho

O arquivo do professor contém divergência de pontuação entre o cabeçalho, os requisitos e a avaliação final. Todos os requisitos funcionais são contemplados, independentemente dessa divergência. Os limites são regras didáticas do desafio, não parâmetros de operação de uma sala de servidores real.

2. Para que serve cada site

Wokwi é o laboratório virtual: simula a placa, o sensor, os LEDs e os fios, compila o programa e executa o circuito. Você pode mudar a temperatura do sensor sem comprar componentes. O trabalho usa o navegador; não é necessário instalar Arduino IDE ou conectar uma placa física.

GitHub guarda e apresenta os arquivos do projeto. O repositório que você criou serve para organizar código, documentação e evidências. Guardar o código no GitHub não cria automaticamente uma simulação no Wokwi.

Monitor serial é a área de texto na simulação. Ela mostra o que o ESP32 imprime com `Serial.print()` e `Serial.println()`. Neste desafio, essa saída representa a telemetria. Ela não é um painel remoto na nuvem.

3. Arquivos e ordem de leitura

Arquivo/pasta	Finalidade
`sketch.ino`	Programa Arduino/C++ executado no ESP32
`diagram.json`	Componentes, posições e conexões do circuito
`libraries.txt`	Biblioteca externa do DHT22
`README.md`	Visão geral, links e instruções rápidas
`docs/GUIA_COMPLETO.md`	Este passo a passo e explicação do código
`docs/ENTREGA.md`	Registro do desafio e respostas das perguntas
`evidencias/`	Capturas reais dos três cenários

Somente os três primeiros arquivos precisam estar no editor do Wokwi. Os demais são documentação no GitHub. O JSON descreve a montagem, e não é o programa da placa. Os nomes dos arquivos importam: mantenha as extensões `.ino`, `.json` e `.txt`.

4. Primeiro acesso e interface do Wokwi

1. Acesse wokwi.com e use **Sign in** para entrar na sua conta. Se precisar criar uma conta, conclua pessoalmente o cadastro e os termos.
2. Abra o modelo Arduino para ESP32. Ele pode abrir com ESP32 DevKitC V4; substituir o `diagram.json` pelo deste projeto seleciona **ESP32 DevKit V1**, exigido no desafio.
3. À esquerda fica o editor; à direita, o circuito. A aba **sketch.ino** contém o programa. **diagram.json** contém a montagem. **Library Manager** contém as bibliotecas adicionadas.
4. O botão verde com triângulo inicia a simulação. Enquanto está rodando, o botão de parada interrompe a execução. Reiniciar executa `setup()` novamente.
5. O botão azul **+** do circuito adiciona componentes. O menu de três pontos oferece controles de visualização, como zoom.
6. **Save** salva o projeto; dê um nome como “Desafio 16 - Monitoramento de temperatura ESP32”. **Share** permite obter o endereço para compartilhar.

O login precisa estar no perfil do navegador em que o Wokwi foi aberto. Estar conectado ao Google em outro perfil do Chrome não significa estar conectado ao Wokwi nesta aba. Salve antes de fechar a página.

5. Carregar este projeto sem montar fio por fio

Esta é a forma de reproduzir exatamente o circuito entregue.

1. Abra o arquivo local `sketch.ino` em um editor de texto, selecione tudo e copie. No Wokwi, abra a aba **sketch.ino**, clique dentro do código, selecione tudo com `Ctrl+A` e cole. A seleção deve estar no editor, não na página inteira.
2. Repita o procedimento com o conteúdo de `diagram.json` na aba correspondente. O ESP32 DevKit V1, a protoboard, o DHT22, os três LEDs, os resistores e os fios devem aparecer.
3. Abra **Library Manager**, clique no **+**, pesquise **DHT sensor library for ESPx** e selecione a biblioteca. Isso cria `libraries.txt`.

4. Para reproduzir a versão utilizada, abra `libraries.txt` e mantenha `DHT sensor library for ESPx@1.19.0` como única linha de biblioteca. Se a aba não aparecer, procure o arquivo no menu das abas.
5. Clique em **Save**. Aguarde a confirmação do salvamento e confira se o endereço passou a conter um número de projeto.
6. Clique no triângulo verde. Na primeira execução, o Wokwi compila e resolve a biblioteca. Aguarde a conclusão; não confunda compilação com a leitura do sensor.

`WiFi.h` vem com o suporte do ESP32 e não precisa ser instalada pelo Library Manager. A biblioteca adicionada fornece `DHTesp.h`; não troque só o nome do `#include` por outra biblioteca, pois as funções também são diferentes.

6. Componentes e conceitos de eletrônica

ESP32 DevKit V1: placa com microcontrolador e Wi-Fi. Executa o programa e controla os pinos. **GPIO** é um pino que o programa pode usar para entrada ou saída. GPIO 18, por exemplo, aparece na placa como D18; o número não é a posição física contando os pinos.

DHT22: sensor digital de temperatura e umidade. Neste trabalho usamos apenas temperatura. Seus pinos são VCC, SDA, NC e GND. VCC é alimentação; SDA é a linha de dados; NC fica sem ligação; GND é a referência elétrica comum. O nome SDA aqui não significa que este sensor use o protocolo I2C.

LED: saída visual, com polaridade. A é o ânodo e C é o cátodo. Neste circuito, o GPIO fornece o sinal pelo resistor até A; C retorna ao GND. Inverter a polaridade pode impedir o LED de acender.

Resistor de 220 ohms: limita a corrente do LED. Cada LED tem seu próprio resistor em série. Ele pode estar antes ou depois do LED desde que esteja no mesmo caminho de corrente. Não o ligue em paralelo com o LED.

Protoboard: permite organizar ligações sem solda. Os trilhos de alimentação são usados para distribuir 3,3 V e GND. Os grupos de cinco furos da área central são ligados entre si; o canal central separa os dois lados. Na montagem entregue, os trilhos inferiores distribuem a alimentação do sensor e o retorno dos LEDs. Os fios explícitos de `diagram.json` conectam os componentes a esses trilhos.

As cores dos fios ajudam a ler a montagem: vermelho para alimentação, preto para GND, azul para os dados do DHT22 e cores dos LEDs para seus sinais. A cor do fio não determina sua função elétrica.

7. Ligações utilizadas

Origem	Destino	Função
ESP32 3V3	Trilho inferior positivo da protoboard	Distribuir alimentação de 3,3 V
ESP32 GND	Trilho inferior negativo da protoboard	Referência comum
Trilho positivo	DHT22 VCC	Alimentar sensor
Trilho negativo	DHT22 GND	Retorno do sensor
DHT22 SDA	ESP32 GPIO 15	Dados de temperatura
DHT22 NC	Sem ligação	Pino não utilizado
ESP32 GPIO 18	Resistor 220 ohms -> LED verde A	Saída NORMAL
LED verde C	Trilho negativo	Retorno do LED verde
ESP32 GPIO 19	Resistor 220 ohms -> LED amarelo A	Saída ATENÇÃO
LED amarelo C	Trilho negativo	Retorno do LED amarelo
ESP32 GPIO 21	Resistor 220 ohms -> LED vermelho A	Saída CRÍTICO
LED vermelho C	Trilho negativo	Retorno do LED vermelho

O JSON usa `bb:bp.1` para o trilho positivo inferior e `bb:bn.1` para o negativo inferior. Outros números nesses mesmos trilhos são pontos de ligação à mesma alimentação. As ligações invisíveis ao `$serialMonitor` apenas conectam o monitor serial virtual ao ESP32.

Para montar manualmente como exercício: adicione os componentes pelo **+**, configure três resistores de 220 ohms e as três cores dos LEDs. Passe o mouse nos pinos para ver seus nomes. Clique no pino inicial, trace o fio e clique no pino final. Mantenha os GPIOs da tabela ou altere tanto o circuito quanto as constantes do programa. Ao montar hardware físico, confira o modelo do DHT22 e sua necessidade de resistor de pull-up; a validação deste projeto é no simulador.

8. Como o código funciona

O fluxo é: **medir -> classificar -> acender um LED -> imprimir os dados -> repetir**.

Bibliotecas e constantes

`#include <DHTesp.h>` carrega as funções do sensor. `#include <WiFi.h>` carrega o suporte de rede. `math.h` fornece `isnan()`, utilizado para detectar leitura inválida.

As constantes `PINO_DHT`, `LED_VERDE`, `LED_AMARELO` e `LED_VERMELHO` documentam os GPIOs. `INTERVALO_LEITURA_MS = 2000` significa 2.000 milissegundos, ou 2 segundos. `DISPOSITIVO` define `ESP32_SALA_01`, que aparece na telemetria.

`DHTesp dht` cria o objeto usado para conversar com o sensor. A temperatura é `float`, pois pode ter casas decimais, como 27,5 °C. Os valores impressos usam ponto decimal, como `27.50 C`.

setup(): configuração inicial

`setup()` roda uma vez a cada início ou reinício. `Serial.begin(115200)` inicia a comunicação serial. `pinMode(..., OUTPUT)` configura os LEDs como saídas. `desligarLeds()` inicia com todas apagadas. `dht.setup(PINO_DHT, DHTesp::DHT22)` informa o GPIO e o modelo do sensor.

O Wi-Fi usa modo estação e a rede `Wokwi-GUEST`, sem senha. O terceiro parâmetro de `WiFi.begin(..., 6)` informa o canal da rede virtual e evita a etapa de busca. A conexão ocorre sem um laço de espera que paralise o sensor.

loop(): repetição e tempo

`loop()` roda continuamente. `verificarWifi()` detecta se a conexão mudou e imprime a mensagem de conexão e o IP. A função `millis()` informa o tempo desde o início do programa.

Quando `agora - ultimaLeitura >= 2000`, o programa atualiza o momento da última leitura e chama `atualizarMonitoramento()`. A subtração com `unsigned long` mantém a comparação adequada mesmo quando o contador dá a volta. A pequena pausa `delay(10)` ajuda a execução; ela não é o intervalo de leitura.

O intervalo é medido pelo relógio da simulação. Se ela estiver lenta, 2 segundos simulados podem levar mais tempo no relógio do computador. Pequenas diferenças de poucos milissegundos na saída são esperadas pela execução do laço e do sensor.

Decisão com if / else if / else

Primeiro o programa apaga os três LEDs. Depois testa a temperatura. A condição `< 25.0` acende verde. Se ela for falsa, a condição `<= 30.0` acende amarelo: o teste anterior já excluiu os valores menores que 25. O `else` restante acende vermelho.

Por isso, 25 °C e 30 °C pertencem à faixa amarela. Usar `>= 30` para o vermelho seria incorreto para este enunciado. A cadeia de decisões e o desligamento prévio garantem um único LED aceso nas leituras válidas.

Uma leitura `NaN` não representa temperatura válida. Nesse caso, o programa apaga os LEDs e imprime `ERRO_SENSOR` e `LED Ativo: NENHUM`. É um tratamento adicional de falha; não é uma quarta faixa de temperatura do desafio.

Telemetria

`Serial.print()` escreve sem pular linha. `Serial.println()` escreve e pula linha. `Serial.print(temperatura, 2)` mostra duas casas decimais. Os textos `ATENCAO` e `CRITICO` sem acentos correspondem aos estados ATENÇÃO e CRÍTICO do enunciado e seguem o exemplo de saída dele.

Além dos campos exigidos, a saída mostra o estado do Wi-Fi e o tempo em milissegundos. Esses campos ajudam a verificar conexão e intervalo de atualização. Ter IP e Wi-Fi conectado não significa que as temperaturas estão sendo armazenadas na internet.

9. Executar os três testes obrigatórios

1. Inicie a simulação. Aguarde a primeira leitura e confira as mensagens iniciais do Wi-Fi.
2. Clique no DHT22 enquanto a simulação estiver rodando. Aparece o controle de temperatura e umidade. Altere **Temperature**; não altere apenas **Humidity**.
3. Configure **20 °C**. Aguarde a próxima leitura de 2 segundos. Deve acender somente o verde, com `Temperatura: 20.00 C`, `Status: NORMAL` e `LED Ativo: VERDE`.
4. Configure **28 °C** sem mudar o código. Aguarde a atualização. Deve apagar verde, acender somente amarelo e imprimir `28.00 C`, `ATENCAO` e `AMARELO`.

5. Configure **35 °C**. Aguarde a atualização. Deve apagar amarelo, acender somente vermelho e imprimir `35.00 C`, `CRITICO` e `VERMELHO`.
6. Volte para 20 °C e verifique que o sistema retorna ao verde. A lógica vale tanto ao subir quanto ao descer a temperatura.

O código não fixa a temperatura; ele consulta o sensor. O valor inicial de 20 °C em `diagram.json` só define a condição inicial da simulação.

10. Testes adicionais dos limites

Entrada	Resultado esperado
24,9 °C	Verde / NORMAL
25,0 °C	Amarelo / ATENCAO
30,0 °C	Amarelo / ATENCAO
30,1 °C	Vermelho / CRITICO

Esses testes complementam os três obrigatórios. Se o controle só permitir números inteiros, teste 24, 25, 30 e 31 °C. Confira também que dois registros consecutivos apresentam aproximadamente 2.000 ms de diferença no campo `Tempo`.

11. Capturar e identificar as evidências

As capturas incluídas foram registradas diretamente na simulação. Para reproduzir, em cada uma das três temperaturas obrigatórias, enquadre o circuito, o LED aceso e o bloco completo da leitura atual no monitor serial. Se possível, inclua também o controle do DHT22 e a identificação do projeto. Aguarde a atualização antes da captura; um LED novo acompanhado de texto antigo não demonstra corretamente o cenário.

Use estes nomes:

- `01-20C-normal.jpg`
- `02-28C-atencao.jpg`
- `03-35C-critico.jpg`

Para fazer novas capturas no Windows, use `Win+Shift+S`, recorte a região necessária e salve em PNG ou JPG. Organize na pasta `evidencias`. Não substitua capturas por imagens ilustrativas: elas devem mostrar a simulação funcionando.

Para demonstrar o extra, confirme `Wi-Fi conectado a Wokwi-GUEST!` na inicialização e `Wi-Fi: CONECTADO` nas leituras. Se o bloco inicial sair da área visível, reinicie a simulação ou use a rolagem do monitor serial.

12. Salvar e obter o link público

1. Clique em **Save** após terminar o código e a montagem. Defina um título claro e confirme o salvamento.
2. Clique em **Share** e copie o link do projeto. O endereço deve ser do tipo `https://wokwi.com/projects/NUMERO`; não entregue o endereço do modelo `/projects/new/esp32`.
3. Abra esse link em uma janela anônima ou em um navegador sem login. Verifique se o circuito e o código aparecem e se o botão de simulação permite executar.
4. Atualize o link no README e no registro de entrega quando modificar ou recriar o projeto.

O professor precisa do link funcional do **Wokwi**. O link do **GitHub** é complementar e ajuda a apresentar o código, o guia e as evidências.

13. Organização no GitHub

Repositório: DouglasBohmer/Monitoramento-Esp32-Wokwi.

Pelo navegador, abra o repositório e use **Add file -> Upload files** (ou a opção de enviar arquivos exibida quando o repositório está vazio). Envie os arquivos extraídos do pacote, preservando as pastas, e finalize com **Commit changes**. Não envie apenas o ZIP se quiser que o código e o README apareçam diretamente no repositório. Se precisar atualizar, substitua os arquivos alterados e faça outro commit.

Se você usa Git, os comandos equivalentes são `git clone`, copiar os arquivos para a cópia, `git add .`, `git commit -m "Implementa Desafio 16"` e `git push`. Um commit registra uma versão; push envia essa versão ao GitHub. Não misture este projeto com repositórios dos seus trabalhos diários.

14. Respostas das perguntas do professor

Pergunta 1 - Diferença entre sensor e atuador: um sensor mede uma grandeza do ambiente e fornece essa informação ao sistema. Um atuador transforma um comando do sistema em uma ação ou saída perceptível. Neste projeto, o DHT22 é o sensor de temperatura; os LEDs verde, amarelo e vermelho são os atuadores de sinalização visual. O ESP32 processa as leituras e decide qual LED acender.

Pergunta 2 - Monitorar remotamente pela internet: seria necessário acrescentar o envio das leituras a um serviço remoto, por exemplo com MQTT ou HTTP/HTTPS, além de um servidor ou plataforma para receber os dados e um painel para consultá-los. Uma implementação real precisaria também de autenticação, proteção da comunicação, armazenamento quando necessário e tratamento de falhas. O Wi-Fi do desafio fornece conectividade; o monitor serial atual exibe a telemetria localmente e não envia os dados para a nuvem.

15. Roteiro curto para explicar em sala

“O DHT22 mede a temperatura e envia o valor ao ESP32 pelo GPIO 15. A cada 2 segundos, o programa lê o sensor, apaga os três LEDs e escolhe a saída conforme a faixa. Abaixo de 25 graus, verde; de 25 a 30 inclusive, amarelo; acima de 30, vermelho. O monitor serial informa o dispositivo, a temperatura, o estado e o LED ativo. O ESP32 também se conecta ao Wokwi-GUEST, mas ainda não publica os dados na nuvem.”

Depois demonstre 20, 28 e 35 °C clicando no DHT22. Prepare-se para explicar: por que há um resistor por LED; por que 30 °C é amarelo; por que `setup()` roda uma vez e `loop()` repete; e por que Wi-Fi conectado não basta para monitoramento remoto.

16. Problemas comuns e soluções

Sintoma	O que conferir
`DHTesp.h: No such file or directory`	Biblioteca e conteúdo de `libraries.txt`
`ERRO_SENSOR` ou temperatura inválida	VCC, GND, SDA no GPIO 15 e modelo DHT22
LED não acende	A/C, resistor em série, GND comum e GPIO correto
Dois LEDs ficam acesos	Código atualizado e ausência de ligações que unam as saídas
30 °C acende vermelho	A condição amarela precisa ser `<= 30.0`
Mudança não aparece imediatamente	Aguarde a próxima leitura de 2 segundos simulados
Monitor serial não aparece	Simulação iniciada, `Serial.begin`, ligações virtuais e saída impressa
Wi-Fi não conecta	Rede `Wokwi-GUEST`, senha vazia, WiFi.h e execução da simulação
Save pede login	Conecte a conta no mesmo perfil do navegador
Link mostra modelo vazio	Salve e copie o link numerado do projeto
Mudanças locais não aparecem no Wokwi	Recole os arquivos ou edite o projeto online e salve
Simulação demora a iniciar	Confira mensagens de compilação e a conexão do navegador

17. Checklist de entrega

- [ ] Seu nome completo e data conferidos no registro de entrega.
- [ ] ESP32 DevKit V1, DHT22, protoboard, três LEDs e três resistores de 220 ohms.
- [ ] Leitura automática do sensor a cada 2 segundos.
- [ ] Apenas um LED aceso em cada leitura válida.
- [ ] Status e LED corretos nas faixas, incluindo 25 e 30 °C.
- [ ] Monitor serial mostra dispositivo, temperatura, status e LED ativo.
- [ ] Três capturas de 20, 28 e 35 °C legíveis.
- [ ] Extra: Wi-Fi conectado e mensagem de conexão demonstrada.
- [ ] Código com GPIOs identificados, comentários, setup(), loop() e if/else.
- [ ] Link Wokwi salvo e acessível sem login.
- [ ] Respostas das duas perguntas incluídas.
- [ ] Links e evidências enviados pessoalmente ao professor pelo canal da disciplina.

18. Referências oficiais

Documentação consultada em 14/10/2026. Os detalhes de interface podem mudar.

Simulação ESP32 | DHT22 e temperatura | Bibliotecas | Rede Wi-Fi | diagram.json | Monitor serial | API DHTesp

Os requisitos da atividade vêm do PDF “Desafio 16 - Monitoramento inteligente de temperatura com ESP32 no Wokwi”, fornecido pelo aluno, da Católica de Santa Catarina, 2026, Prof. Msc. Rodrigo Hagstrom.

Apêndice: código completo

O mesmo programa está no arquivo sketch.ino e na simulação pública. Os comentários explicam as principais rotinas.

```
/*
 * Desafio 16 - Monitoramento de uma sala de servidores
 * ESP32 DevKit V1 + DHT22 + LEDs + Wi-Fi no Wokwi.
 * O monitor serial representa a telemetria local deste desafio.
 */

#include <DHTesp.h>
#include <WiFi.h>
#include <math.h>

// GPIOs: estes numeros devem coincidir com as ligacoes do diagram.json.
const int PINO_DHT = 15;
const int LED_VERDE = 18;
const int LED_AMARELO = 19;
const int LED_VERMELHO = 21;

const unsigned long INTERVALO_LEITURA_MS = 2000;
const char* DISPOSITIVO = "ESP32_SALA_01";
const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_SENHA = "";

DHTesp dht;

unsigned long ultimaLeitura = 0;
bool wifiEstavaConectado = false;

// Desliga todos antes de escolher um: impede dois LEDs acesos juntos.
void desligarLeds() {
  digitalWrite(LED_VERDE, LOW);
  digitalWrite(LED_AMARELO, LOW);
  digitalWrite(LED_VERMELHO, LOW);
}

// A conexao ocorre em paralelo; o sensor nao depende do Wi-Fi.
void verificarWifi() {
  bool conectado = WiFi.status() == WL_CONNECTED;
  if (conectado && !wifiEstavaConectado) {
    Serial.println("Wi-Fi conectado a Wokwi-GUEST!");
    Serial.print("Dispositivo: ");
    Serial.println(DISPOSITIVO);
    Serial.print("Endereco IP: ");
    Serial.println(WiFi.localIP());
  } else if (!conectado && wifiEstavaConectado) {
    Serial.println("Wi-Fi desconectado. Monitoramento local continua.");
  }

  wifiEstavaConectado = conectado;
}

// Le o sensor, aplica as faixas do enunciado e imprime a telemetria.
void atualizarMonitoramento() {
  float temperatura = dht.getTemperature();
  const char* status;
  const char* ledAtivo;
  desligarLeds();

  // Uma falha nao pode ser apresentada como temperatura normal ou critica.
  if (isnan(temperatura)) {
    status = "ERRO_SENSOR";
    ledAtivo = "NENHUM";
  } else if (temperatura < 25.0) {
    status = "NORMAL";
    ledAtivo = "VERDE";
    digitalWrite(LED_VERDE, HIGH);
  } else if (temperatura <= 30.0) {
    // O primeiro if ja excluiu os valores abaixo de 25 C.
```

```
065      status = "ATENCAO";
066      ledAtivo = "AMARELO";

067      digitalWrite(LED_AMARELO, HIGH);
068    } else {
069      status = "CRITICO";
070      ledAtivo = "VERMELHO";
071      digitalWrite(LED_VERMELHO, HIGH);
072    }
073
074    Serial.println("\nSISTEMA DE MONITORAMENTO IoT");
075    Serial.print("Dispositivo: ");
076    Serial.println(DISPOSITIVO);
077    Serial.print("Temperatura: ");
078    if (isnan(temperatura)) {
079      Serial.println("LEITURA INVALIDA - confira o DHT22");
080    } else {
081      Serial.print(temperatura, 2);
082      Serial.println(" C");
083    }
084    Serial.print("Status: ");
085    Serial.println(status);
086    Serial.print("LED Ativo: ");
087    Serial.println(ledAtivo);
088    Serial.print("Wi-Fi: ");

089    Serial.println(WiFi.status() == WL_CONNECTED ? "CONECTADO" : "CONECTANDO/SEM CONEX
     AO");
090    Serial.print("Tempo: ");
091    Serial.print(millis());
092    Serial.println(" ms");
093    Serial.println("----------------------------------------");
094  }
095
096  // Executada uma vez ao iniciar ou reiniciar a simulacao.
097  void setup() {
098    Serial.begin(115200);
099    pinMode(LED_VERDE, OUTPUT);
100    pinMode(LED_AMARELO, OUTPUT);
101    pinMode(LED_VERMELHO, OUTPUT);
102    desligarLeds();
103    dht.setup(PINO_DHT, DHTesp::DHT22);
104
105    Serial.println("SISTEMA DE MONITORAMENTO IoT - INICIANDO");
106    Serial.print("Dispositivo: ");
107    Serial.println(DISPOSITIVO);
108    Serial.println("Conectando ao Wi-Fi Wokwi-GUEST (sem senha)...");
109    WiFi.mode(WIFI_STA);
110    WiFi.setAutoReconnect(true);

111    WiFi.begin(WIFI_SSID, WIFI_SENHA, 6);
112    ultimaLeitura = millis();
113  }
114
115  // Repete continuamente. millis() mede o intervalo sem bloquear o Wi-Fi.
116  void loop() {
117    verificarWifi();
118    unsigned long agora = millis();
119    if (agora - ultimaLeitura >= INTERVALO_LEITURA_MS) {
120      ultimaLeitura = agora;
121      atualizarMonitoramento();
122    }
123    delay(10); // Pequena pausa para o simulador; nao define a cadencia do sensor.
124  }
```